and used a real PostgreSQL database both for development and testing, and it's the same database we want to use in production.

Moving from a PostgreSQL container running in your local environment to a managed service with high availability, scalability, and resilience is a matter of changing the values of a few configuration properties for Spring Boot. How great is that?

First, create a new PostgreSQL server named `polar-postgres`, as shown in the following code snippet. We'll use PostgreSQL 14, which is the same version we used for development and testing. Remember to replace `<your_region>` with the geographical region you'd like to use. It should be the same as the region you used for the Kubernetes cluster. In my case, it's `ams3`:

```
$ doctl databases create polar-db \
    --engine pg \
    --region <your_region> \
    --version 14
```

The database server provisioning will take several minutes. You can verify the installation status with the following command (I have filtered the result for the sake of clarity):

```
$ doctl databases list

ID               Name        Engine    Version    Region    Status
<polar-db-id>    polar-db    pg        14         ams3      online
```

When the database is `online`, your database server is ready. Take note of the database server ID. You'll need it later.

To mitigate unnecessary attack vectors, you can configure a firewall so that the PostgreSQL server is only accessible from the Kubernetes cluster created